

NEXUS Portal FCE

Informe de estado técnico

Plataforma de aprendizaje adaptativo · Cátedra de Economía I · UNPSJB FCE

Versión v19.30.7 · 19 de abril de 2026

Responsable técnico: Juan (desarrollo solo)

Destinatario: auditoría académica

Estado del proyecto: portal desplegado en producción. Demo institucional ejecutada el 18 de abril de 2026.

0. Observaciones estratégicas y lectura de valor

Esta sección complementa el informe técnico con una capa explícita de interpretación estratégica, orientada a perfiles no técnicos (dirección académica, responsables institucionales, potenciales inversores). El objetivo es que el estado actual del sistema pueda leerse no solo como un conjunto de decisiones de arquitectura, sino como una propuesta de valor verificable.

0.1 Propuesta de valor

Más allá de su implementación técnica, el portal NEXUS aporta ventajas operativas concretas para la cátedra y para la experiencia de estudio:

- **Reducción del tiempo de estudio** mediante resolución inmediata de consultas en lenguaje natural.
- **Mejora de la calidad del aprendizaje** al evitar respuestas incorrectas o alucinadas, gracias a un control activo en tres capas.
- **Escalabilidad del contenido** sin necesidad de rediseñar la arquitectura.
- **Funcionamiento parcialmente offline** por su naturaleza PWA (service worker con caché versionado).
- **Potencial de extensión** a otras materias o contextos institucionales, preservando el mismo núcleo adaptativo.

0.2 Reposicionamiento del estado actual

El sistema debe leerse no como un prototipo experimental, sino como una plataforma en etapa de expansión. Tres datos lo respaldan y quedan reflejados en el Resumen ejecutivo (sección 1):

- **El sistema ya es funcional y está validado en condiciones reales:** desplegado en producción sobre Vercel, con la base de conocimiento activa, la demo institucional ejecutada el 18 de abril de 2026 y el simulacro técnico verde al 100 %.
- **El riesgo técnico se encuentra acotado y controlado:** no hay entries de severidad ALTA o MEDIA pendientes, los cambios recientes son 100 % generation-time y el runtime quedó intocado (hash de la KB idéntico pre/post).
- **El roadmap restante apunta a escala y experiencia de usuario, no a resolver fallas estructurales:** Fase 3 amplía el corpus (OCR Itinerario I) y Fase 4 integra la UI de la presentación al portal productivo.

0.3 Valor diferencial — arquitectura de confiabilidad del conocimiento

La existencia de una arquitectura de confiabilidad del conocimiento compuesta por tres capas es la ventaja competitiva central del sistema frente a soluciones genéricas basadas únicamente en modelos de lenguaje:

- **Validator estructural (Fase 2):** campos obligatorios, frases prohibidas, deduplicación semántica.
- **Auditoría heurística independiente (H1-H7):** revisa todo el KB con reglas ortogonales al generador.
- **Validator semántico bloqueante (S1-S5):** detiene contenido alucinado en el momento de generación, antes de que llegue al KB.

Este triple control reduce significativamente el riesgo de respuestas incorrectas y sostiene la credibilidad del portal como herramienta académica, incluso cuando se amplía el corpus con nuevos autores y unidades.

0.4 Traducción funcional de la Fase 4 (UI Sovereign)

La integración de la nueva interfaz no es solamente un cambio estético: su impacto funcional se traduce en dimensiones medibles desde el punto de vista del usuario y de la institución.

- **Engagement del usuario:** interacciones más fluidas, feedback inmediato y elementos de juego (ticker, modo pánico, toast de patrones) que sostienen la atención.
- **Retención y continuidad de estudio:** la cicladora de ejercicios y el motor adaptativo empujan a completar ciclos en lugar de abandonar.
- **Percepción de calidad del sistema:** el design system premium equipara la experiencia visual a la de productos reconocidos del sector (Notion, Duolingo, Stripe), lo cual influye directamente en la adopción.
- **Diferenciación frente a plataformas educativas tradicionales:** un portal de cátedra deja de sentirse como un sitio estático de apuntes para pasar a comportarse como una herramienta activa de estudio.

0.5 Riesgos controlados

Se documentan de forma explícita los riesgos actuales, cada uno con su mecanismo de mitigación ya instalado:

- **Generación de contenido aún dependiente de un pipeline manual** → mitigado por el validator semántico bloqueante (Fase 2.5), que evita que contenido alucinado llegue al KB.
- **Cache QA no activado** → scaffolding pasivo ya presente en el repositorio (kb/qa_cache.json), con activación planificada y reversible para la Fase 3.

- **Convivencia temporal de dos UIs durante la migración** → controlada mediante feature flag reversible (window.NEXUS_UI) y paridad funcional verificable con doble test harness antes de cada merge.

0.6 Conclusión estratégica

En su estado actual, NEXUS no debe considerarse un prototipo experimental, sino un sistema funcional con:

- Base de conocimiento validada y trazable al material de cátedra.
- Mecanismos activos de control de calidad operando en producción.
- Arquitectura preparada para escalar contenido y mejorar la experiencia de usuario.

El foco de las próximas fases no es corregir el sistema, sino expandirlo y optimizar su adopción.

1. Resumen ejecutivo

El portal NEXUS es una PWA desplegada sobre Vercel que sirve material didáctico indexado, consultas en lenguaje natural con matcher híbrido (KB local + LLM vía proxy), y un motor adaptativo de práctica. La presentación institucional del 18 de abril de 2026 utilizó un preview standalone con design system premium distinto al portal productivo.

Lectura del estado actual (sintetizada desde la sección 0.2)

- El sistema **ya es funcional y está validado en condiciones reales** — deployado, con demo institucional ejecutada y simulacro técnico 10/10 verde.
- El **riesgo técnico está acotado y controlado** — cero entries ALTA/MEDIA pendientes, runtime intocado en los cambios recientes.
- El **roadmap remanente apunta a escala y UX**, no a resolver fallas estructurales.

Este informe documenta:

- Lo que está hecho y verificado (Fases 1, 2 y 2.5 del roadmap).
- Lo que queda por hacer (Fases 3 y 4).
- Cómo se integrará la UI de la presentación al portal productivo de forma segura (Fase 4, nueva).
- Cambios técnicos adicionales detectados durante el hardening y la estrategia propuesta para cada uno.

Estado sintético

- Portal productivo deployado — 138 entries en la base de conocimiento.
- Simulacro de 10 queries críticas: 10/10 verde (latencias 6–46 ms).
- Tres capas de control de alucinación instaladas (validator básico, audit H1–H7, semantic validator S1–S5).
- Cobertura de materias: Contabilidad, Administración, Sociales, Propedéutica.
- Materiales pendientes de ingesta: Itinerario I de Contabilidad (2 PDFs, 37 páginas) — pipeline OCR validado y listo para ejecutar post-demo.
- Integración UI presentada: diseño y roadmap detallado en la sección 5.

2. Arquitectura resumida

Frontend: index.html (PWA monolítica, 5 162 líneas), portal.js, nexus-ai.js (matcher híbrido), nexus-adaptive-engine.js (motor adaptativo), service worker sw.js con caché versionado.

Base de conocimiento: kb/knowledge_base.json — 138 entries con schema 1.0, cada una con id, type, patterns[], answer_full, source_refs[], materia, unidad.

Pipeline de generación (offline)

- ingest_pdf.py — extrae texto de PDFs del corpus (Materiales/).
- generate_kb.py — genera drafts con Mistral 7B local vía Ollama.
- validator.py — valida y consolida drafts contra el KB productivo.

Runtime: matcher fuzzy (Jaccard + Levenshtein) con umbrales de confianza 0.85 (hit directo) / 0.60 (hit tentativo re-escrito por Llama 3.2) / <0.60 (fallback a LLM vía proxy).

Deploy: Vercel autodeploy desde main + verificación remota con scripts/verify_deploy.py.

3. Lo hecho — Fases 1, 2 y 2.5

3.1 Fase 1 — Corpus, ingesta y matcher

Estado: completa, en producción.

- Ingesta de 4 PDFs núcleo del corpus (Bourdieu, Chiavenato ×2, Itinerario II y III de Contabilidad).
- 138 entries generadas, deduplicadas, con referencias a página verificables.
- Matcher runtime operativo con latencia media <15 ms sobre el KB completo.
- Proxy LLM para fallback cuando el matcher no encuentra hit.

Evidencia de salud: verify_deploy.py --local verde; demo_simulacro.js 10/10 verde.

3.2 Fase 2 — Validator básico y deduplicación

Implementado en pipeline/validator.py. Reglas de aceptación:

- Campos obligatorios presentes (id, type, patterns ≥ 2, answer_full ≥ 80 caracteres).
- Frases prohibidas anti-alucinación / anti-evasión filtradas (FORBIDDEN_PHRASES).
- source_refs no vacío para tipos material_concept y material_example.
- Deduplicación de patterns inter-entry (gana la primera).
- Deduplicación semántica de answers por Jaccard de tokens (umbral 0.60).
- Auto-fix de sufijos en inglés (_concept → _concepto, etc.).
- Resolución de colisiones de ID con sufijo incremental.

3.3 Auditoría independiente — script audit_hallucinations.py

Durante la revisión se detectó que el generador (Mistral 7B) producía entries con tres clases de problema no capturables por el validator básico:

- Fugas cross-domain — una entry de Sociales que mezclaba conceptos contables en su definición.
- IDs genéricos y duplicados semánticos — bourdieu_concepto_3, _5, _7 con contenido redundante.
- Typos alucinados — Mistral tipeó "marcado" por "mercado" y generó contenido incoherente que, por cercanía Levenshtein, matcheaba fuzzy con la query real del usuario.

Se construyó scripts/audit_hallucinations.py con siete heurísticas independientes:

Heurística	Detecta	Severidad
H1	Mención a materia ajena	alta si combina con marker
H2	ID genérico con sufijo numérico _N	baja aislada
H3	Todos los patterns son templates formulaicos	media si combina con H2
H4	Autor esperado no aparece en el contenido	media
H5	Answer demasiado breve	baja
H6	Frases prohibidas	alta
H7	Raíz del ID a distancia Levenshtein ≤ 2 de un término común sin match exacto	alta si combina con H3

Resultado del triage previo al demo: sobre 138 entries, se flaguaron 44 (31,9 %). Se aplicó cirugía textual sobre 4 (Admin, limpieza cross-domain conservadora) y neutralización (patterns=[], demo_safe=false, backup en _original_patterns) sobre 21 adicionales (Sociales + Admin). Dos falsos positivos fueron explícitamente whitelisted.

Estado final de la base de conocimiento

Métrica	Valor
Entries activas (servidas por el matcher)	115
Entries neutralizadas (presentes para trazabilidad, invisibles al runtime)	23
Entries whitelisted (audit_whitelist_reason documentado)	2
Entries severidad ALTA o MEDIA pendientes	0
Entries severidad BAJA pendientes (code-smell H2/H3)	21

3.4 Fase 2.5 — Semantic validator (completada 2026-04-19)

La auditoría identificó que las tres clases de problema podían repetirse en cada batch nuevo (OCR Itinerario I, Foucault, Fisher, etc.). Para prevenir que contenido alucinado llegue al KB, se extendió pipeline/validator.py con cinco reglas bloqueantes:

Regla	Detecta	Espeja
S1 cross_domain_leak	Materia ajena mencionada + marker fuerte de esa materia en el mismo answer	H1
S2 overgen_id	ID con sufijo numérico _N donde $N \geq 3$	H2 (crítico)
S3 pattern_diversity	Todos los patterns formulaicos sin keyword específica	H3
S4 author_presence	Entry con autor esperado (vía expected_author o mapping materia+unidad) donde el autor no aparece en el contenido	H4
S5 typo_hallucination	Raíz del ID a distancia Levenshtein ≤ 2 de un término común sin match exacto	H7

El comportamiento de S1-S5 es reversible: NEXUS_VALIDATOR_STRICT=0 desactiva el bloqueo (útil para migrar drafts legacy). Las entries con validator_whitelist: true o audit_whitelist_reason saltan S1-S5 — el mismo mecanismo de escape que usa la auditoría.

Diferencia de rol entre audit y validator

- **Audit:** reporta todo lo sospechoso (21 entries flagueadas BAJA en KB actual), incluyendo code-smells aceptables.
- **Validator:** bloquea solo lo que es inaceptable en generación nueva (5 entries bloqueadas en la KB actual si se reprocesara — todas ya conocidas por la auditoría).

Verificación de Fase 2.5

Verificación	Resultado
Unit tests internos (S1-S5 + helpers) vía python pipeline/validator.py --self-test	✓ 15 tests verdes
A/B diff STRICT=OFF vs STRICT=ON sobre draft real Chiavenato	+1 S3 catch adicional
Parity check validator $\rightleftharpoons$ audit sobre KB actual	Overlap $V \cap A = 100\% \cdot V\text{-only} = 0$
KB hash pre/post build (md5sum kb/knowledge_base.json)	Idéntico (Fase 2.5 es 100 % generation-time)
verify_deploy.py --local	20/20 controles verdes
demo_simulacro.js (10 queries del DEMO_FLOW)	10/10 verdes · latencias 6-46 ms

4. Lo que queda — Roadmap remanente

4.1 Fase 3 — OCR + cache layer

Estado: documentado en OCR_SCOUTING.md, ejecutable post-demo.

- Pipeline OCR validado: Tesseract 4.1.1 + Poppler, lang spa. Tiempo end-to-end estimado 45–60 min para los 2 PDFs escaneados de Itinerario I de Contabilidad (37 páginas).
- Proyección: la KB pasa de 138 a ~200 entries; Contabilidad pasa de 40 a ~100.
- Cache QA: kb/qa_cache.json ya existe como infra pasiva (feature flag OFF). Activación post-demo.

4.2 Fase 4 — Integración UI Sovereign

Nueva. Detallada en la sección 5.

5. Fase 4 — Integración segura de la UI Sovereign

5.1 Contexto

La UI mostrada en la presentación del 18 de abril de 2026 es un archivo standalone (NEXUS-preview-sovereign.html, 1 525 líneas) con un design system premium propio: tipografía Inter/JetBrains Mono, paleta oscura con acentos verde / púrpura / rojo, cicladora de ejercicios, modo pánico activable, ticker de eventos, modal manifiesto NEXUS_OS y toast de patrones detectados. No comparte CSS ni namespaces con index.html (el portal productivo), que tiene otra arquitectura visual y 5 162 líneas.

Integrar esta UI al portal real es un trabajo que, si se hace mal, rompe la demo. Si se hace bien, eleva el producto al nivel visual que ya vio el auditor. La Fase 4 está diseñada para que la integración sea reversible, progresiva y verificable en cada paso.

5.2 Principios de diseño

- **UI-only.** Ninguna línea de Fase 4 toca el KB, el matcher, el validator, ni el pipeline. El backend permanece estable.
- **Feature flag.** Un solo booleano (window.NEXUS_UI = 'sovereign' | 'legacy') controla la rama de render. Rollback inmediato si algo falla.
- **Doble test harness.** Simulacro y demo_evitar.js deben correr verdes en ambos modos antes de cada merge.
- **Infra pasiva primero, activación al final.** Patrón ya validado en Fase 3 (cache QA scaffolding).
- **Colisión cero de namespaces.** La UI Sovereign se migra bajo prefijo .sov-* para coexistir con las clases actuales (.sb-*, .pw-*) sin side effects cruzados.

5.3 Sub-fases

Fase 4.0 — Audit arquitectónico (estimado: 2-3 h)

- Diff estructural NEXUS-preview-sovereign.html vs index.html.
- Clasificación de elementos en: visual-only (tokens de color, spacing, motion, tipografía), componentes nuevos (cicladora, modo pánico, ticker, manifiesto OS, toast) y componentes existentes con reskin (sidebar, main content, acordeones).
- Inventario de conflictos de namespace: el preview usa clases genéricas (.chip, .dot, .bar, .accent, .card) que colisionan con utilidades de index.html. Documentar y planificar el renombre.
- **Entregable:** FASE4_AUDIT.md con matriz de migración componente-por-componente.

Fase 4.1 — Routing dual con feature flag (estimado: 4-6 h)

- Servir ambas UIs sobre el mismo dominio: default → portal actual; ?ui=sovereign → preview en modo integrado (ya con matcher real, no mock).
- El preview, al ser servido desde la ruta integrada, reemplaza sus datos mock por llamadas reales a nexus-ai.js y kb/knowledge_base.json.
- Flag en localStorage + query param (?ui=sovereign setea cookie para sesión).
- **Entregable:** switch operativo, preview usando el matcher real con las mismas queries del DEMO_FLOW. Simulacro 10/10 verde en ambos modos.

Fase 4.2 — Extracción de design tokens (estimado: 3-4 h)

- Crear nexus-sovereign-tokens.css con el palette completo, spacings, radii, shadows, motion durations, type scale del preview.
- Crear nexus-sovereign-components.css con clases namespaced (.sov-chip, .sov-dot, .sov-card, .sov-cycle-*).
- Cargar los archivos solo si NEXUS_UI === 'sovereign'.
- Verificar que el portal legacy no se ve afectado (no hay filtraciones de especificidad).
- **Entregable:** stylesheet aislado, cargable bajo flag, sin regresión visual en modo legacy.

Fase 4.3 — Migración gradual componente-por-componente (estimado: 8-12 h)

Orden propuesto, de menor a mayor riesgo:

- Chrome visual base — fondo, tipografía, grid overlay.
- Sidebar — reskin con namespace .sov-sb-*, paridad funcional con la sidebar actual.
- Main content container — acordeones de síntesis (Bloque A del NEXUS Content Standard) con estilos sovereign.
- Toast de patrones detectados — nuevo componente, wire con el observer adaptativo existente.
- Ticker de eventos — nuevo componente, wire con el log de eventos del engine.

- Cicladora de ejercicios — nuevo componente, wire con nexus-quiz.js.
- Modo pánico — nuevo componente, feature toggleable (8 s por pregunta, entrenamiento bajo presión).
- Manifiesto NEXUS_OS — overlay modal, contenido editorial.

Cada componente: PR separado, paridad verificada con checklist, simulacro + demo_evitar verde antes del merge.

Fase 4.4 — Switch de default (decisión separada, post-4.3)

- Solo cuando todos los componentes críticos estén migrados, tests verdes, KB intacta.
- Flag flip: NEXUS_UI = 'sovereign' como default. Modo legacy accesible con ?ui=legacy durante al menos un ciclo académico.
- Requiere aprobación explícita (auditor / responsable técnico).

Fase 4.5 — Deprecación de legacy UI

- Cuando la telemetría indique uso residual del flag legacy (<5 % de sesiones durante 14 días).
- Consolidar index.html, bumpear cache del service worker, limpiar dead code.

5.4 Safety net transversal

- **Checkpoint funcional antes de cada merge:** python pipeline/validator.py --self-test + node scripts/build.js + python scripts/verify_deploy.py --local + node demo_simulacro.js. Los cuatro deben ser verdes.
- **Rollback de un comando:** setear window.NEXUS_UI = 'legacy' en consola o flip de la flag en nexus-ai-config.json.
- **Monitoring pre-default:** 48 h de telemetría con label ui_mode=sovereign antes de considerar el flip de la Fase 4.4. Métricas de referencia: tiempo al primer match, tasa de fallback LLM, latencia p95.
- **Estanqueidad cross-fase:** la Fase 4 NO toca pipeline/, kb/, scripts/audit_hallucinations.py. Un bug en 4.x jamás puede contaminar la KB.

5.5 Estimación total

Sub-fase	Horas estimadas	Riesgo
4.0 Audit arquitectónico	2-3	bajo
4.1 Routing dual + flag	4-6	medio
4.2 Design tokens	3-4	bajo
4.3 Migración gradual	8-12	medio

4.4 Switch default	1	medio-alto
4.5 Deprecación legacy	2-3	bajo
Total	20-29 h	

Horizonte realista: 2-3 semanas de esfuerzo part-time, dado que se intercala con Fase 3 (OCR Itinerario I) y mantenimiento ordinario del portal.

6. Cambios técnicos adicionales detectados

Durante el hardening de Fase 2.5 surgieron observaciones que no son estrictamente parte del roadmap pero que cabe documentar.

6.1 Duplicación de constantes audit ⇔ validator (refactor menor)

scripts/audit_hallucinations.py y pipeline/validator.py comparten hoy por copia: COMMON_TERMS (S5 / H7), sufijos canónicos de ID (_id_root), y función Levenshtein.

Propuesta: mover a pipeline/config.py (o a pipeline/_shared.py) e importar desde ambos lados.

Beneficio: imposible que diverjan al evolucionar. Costo: 30 min. Prioridad: media, recomendable antes de Fase 4.

6.2 Las 21 entries BAJA pendientes (code-smell)

Contenido correcto pero forma mejorable (IDs genéricos _concepto_2, patterns todos formulaicos). No afectan la experiencia actual.

Propuesta post-demo: script scripts/regenerate_baja.py que reprocese solo esas 21 con el validator en modo estricto — el validator rechazará lo malo, quedarán los drafts buenos. Estimado: 2 h. Prioridad: baja, sin bloqueo para Fases 3 y 4.

6.3 Script pipeline/clean_ocr.py referenciado pero no creado

En OCR_SCOUTING.md se menciona un paso de limpieza post-OCR (reunir líneas partidas, restaurar símbolo §, remover headers repetidos). Script de ~50 líneas que debe crearse antes de ejecutar la Fase 3. Prioridad: alta (bloquea OCR Itinerario I).

6.4 Accessibility audit del preview Sovereign

El preview tiene ARIA labels en los modals pero no se verificaron focus traps ni navegación con teclado completa.

Propuesta: incluir en la Fase 4.0 (audit arquitectónico) una revisión axe-core sobre el preview antes de migrar. Prioridad: alta (requisito para deploy institucional).

6.5 Ruido de git status en mount del sandbox (no afecta producción)

El entorno de desarrollo remoto muestra todos los archivos del repositorio como "deleted + untracked" simultáneamente debido al symlink behavior del mount sobre Windows. No hay drift real en el repo; los commits y pushes se hacen desde la terminal nativa del responsable técnico. Documentado en la recipe de commit.

6.6 Extensión de EXPECTED_AUTHORS_BY_MATERIA_UNIDAD con futuros autores

El mapping actual cubre Bourdieu (Sociales U3) y Chiavenato (Admin U2 / U6). Al ingestar Foucault, Fisher, Fraser u otros, extender el diccionario (1 línea por autor).

7. Anexos

Anexo A — Comandos de verificación reproducibles

```
# 1. Self-test del validator (Fase 2.5)
python pipeline/validator.py --self-test
# → [validator self-test] ✓ todos los checks S1-S5 + helpers pasaron
```

```
# 2. Audit completo de la KB
python scripts/audit_hallucinations.py \
  --exclude-neutralized --exclude-whitelisted \
  --out HALLUCINATION_AUDIT_pending.md
# → 0 ALTA, 0 MEDIA, 21 BAJA
```

```
# 3. Preflight local
node scripts/build.js
python scripts/verify_deploy.py --local
# → Build completo: 29 archivos · 2449 KB
# → preflight OK
```

```
# 4. Simulacro end-to-end
node demo_simulacro.js
# → 10/10 verdes · 0 fallas
```

Anexo B — Métricas de la KB al 19 de abril de 2026

Métrica	Valor
Entries totales	138
Entries activas (servidas)	115

Entries neutralizadas	23
Entries whitelisted	2
Cobertura por materia	Contabilidad 40 · Administración 42 · Sociales 48 · Propedéutica 8
Audit pending · alta / media / baja	0 / 0 / 21
Simulacro DEMO_FLOW	10/10 verde
Latencia runtime (p50 / p95)	11 ms / 46 ms

Anexo C — Archivos modificados en la sesión de hardening (2026-04-19)

Archivo	Cambio
pipeline/validator.py	+~280 líneas: reglas S1–S5, helpers, self-tests, flag --self-test
scripts/audit_hallucinations.py	Ya contenía H7 + --exclude-* tras iteración previa del día
INFORME_AUDITOR.md	Nuevo, este documento

Anexo D — Archivos de referencia para el auditor

- CLAUDE.md — instrucciones de arquitectura y principios de contenido.
- DEMO_CONTEXT.md — estado del proyecto al arranque de cada sesión.
- DEMO_FLOW.md — las 10 queries verificadas del demo 2026-04-20.
- NEXUS_HYBRID_ARCHITECTURE_v1.2.md — arquitectura del matcher híbrido.
- OCR_SCOUTING.md — plan de ejecución Fase 3.
- HALLUCINATION_AUDIT_pending.md — code-smells pendientes post-demo.

Fin del informe.